



## 课程大作业说明书

课程名： 分布式应用开发技术 (Q)

题 目： SpringBoot-buy

二级学院： 计算机信息工程学院

专 业： 软件工程

班 级： 22 软件一

学号姓名： 22030309 黄哲

22030313 李远志

22030314 刘骐玮

指导教师： 江亮

2025 年 6 月

# springboot-buy

## 摘要

本项目基于 Spring Boot 和 Spring Cloud 技术，设计并实现了一个分布式电商系统。系统采用前后端分离和微服务架构，涵盖用户管理、商品管理、订单管理、内容管理、支付、邮件通知等核心功能模块。通过引入 MyBatis-Plus 简化数据持久层开发，利用 MySQL 进行数据存储，借助 Nacos 实现服务注册与发现，提升了系统的可扩展性和高可用性。项目还集成了 Apifox 等接口测试工具，保障了各模块间的高效联调与稳定运行。系统结构清晰，功能完善，能够满足现代电商平台的基本业务需求，为后续功能扩展和系统优化提供了良好基础。

**关键词：**Spring Boot；Spring Cloud；微服务；电商系统；MyBatis-Plus

# springboot-buy

## Abstract

This project is based on Spring Boot and Spring Cloud technologies, and designs and implements a distributed e-commerce system. The system adopts a front-end and back-end separation as well as a microservices architecture, covering core functional modules such as user management, product management, order management, content management, payment, and email notifications. MyBatis-Plus is introduced to simplify the development of the data persistence layer, while MySQL is used for data storage. Nacos is utilized for service registration and discovery, enhancing the system's scalability and high availability. The project also integrates interface testing tools such as Apifox to ensure efficient integration and stable operation between modules. The system features a clear structure and comprehensive functions, meeting the basic business needs of modern e-commerce platforms and providing a solid foundation for future functional expansion and system optimization.

**Keywords:** Spring Boot; Spring Cloud; Microservices; E-commerce System; MyBatis-PlusX

# 目 录

|              |                    |          |
|--------------|--------------------|----------|
| <b>第 1 章</b> | <b>引言.....</b>     | <b>3</b> |
| 1.1          | 项目目的.....          | 3        |
| 1.2          | 本文组织结构.....        | 3        |
| <b>第 2 章</b> | <b>相关技术概述.....</b> | <b>4</b> |
| 2.1          | 后端开发框架.....        | 4        |
| 2.1.1        | Spring Boot .....  | 4        |
| 2.1.2        | Spring Cloud ..... | 4        |
| 2.2          | 数据持久层.....         | 5        |
| 2.2.1        | MyBatis-Plus.....  | 5        |
| 2.2.2        | MySQL.....         | 5        |
| 2.3          | 服务治理与配置中心.....     | 5        |
| 2.3.1        | Nacos.....         | 5        |
| 2.4          | 接口测试与调试工具.....     | 6        |
| 2.4.1        | Apifox .....       | 6        |
| 2.5          | 本章小结.....          | 6        |
| <b>第 3 章</b> | <b>系统分析设计.....</b> | <b>7</b> |
| 3.1          | 需求分析.....          | 7        |
| 3.2          | 可行性分析.....         | 8        |
| 3.3          | 项目系统架构.....        | 9        |
| 3.4          | 项目设计.....          | 9        |
| 3.4.1        | 主要功能模块设计.....      | 10       |
| 3.4.2        | 数据库设计.....         | 13       |
| 3.5          | 本章小结.....          | 15       |

|              |              |           |
|--------------|--------------|-----------|
| <b>第 4 章</b> | <b>项目测试</b>  | <b>16</b> |
| 4.1          | 主要功能模块测试     | 16        |
| 4.1.1        | buy-user     | 16        |
| 4.1.2        | buy-orders   | 21        |
| 4.1.3        | buy-email    | 25        |
| 4.1.4        | buy-pay      | 26        |
| 4.2          | 本章小结         | 28        |
| <b>第 5 章</b> | <b>总结与展望</b> | <b>29</b> |
| 5.1          | 总结           | 29        |
| 5.1.1        | 工作内容回顾       | 29        |
| 5.1.2        | 难点检讨         | 29        |
| 5.1.3        | 收获心得         | 29        |
| 5.2          | 展望           | 29        |
| 致 谢          |              | 30        |
| 参考文献         |              | 31        |

# 第 1 章 引言

## 1.1 项目目的

本项目旨在开发一个基于 Spring Boot 的分布式电商系统,实现商品管理、订单管理、用户管理、内容管理、支付、邮件通知等核心功能。通过模块化设计和微服务架构,提升系统的可扩展性、可维护性和高可用性,满足实际电商业务的需求。项目目标包括:

提供完整的电商业务流程支持,包括商品浏览、下单、支付、订单跟踪等。

实现用户、商品、订单、内容等数据的分布式管理,提升系统性能和并发处理能力。

通过模块解耦,便于后续功能扩展和系统维护。

集成邮件通知、支付接口等第三方服务,提升用户体验。

采用主流技术栈和开发规范,提升项目的学习和实践价值。

## 1.2 本文组织结构

本文由六个章节组成,其组织结构如下所述:

第一章介绍了本文的研究背景和意义,阐明了开发分布式电商系统的必要性及其实际应用价值。

第二章介绍了本项目所采用的主要技术和相关算法,为后续章节的系统设计与实现奠定了理论基础。

第三章首先提出了系统的需求分析和整体架构设计,随后依次阐述了数据获取流程和各个功能模块的设计工作,并详细说明了每个模块设置的意义、需要解决的具体问题以及具体的实现流程。

第四章首先给出了系统的交互流程设计,之后详细介绍了各模块的具体实现方法。对于重点环节,提供了伪代码和效果图,并对关键技术细节进行了重点阐述。

第五章对所设计和实现的系统进行了测试,先对各功能模块进行单独测试,再进行集成测试,并对每项测试结果进行了展示与分析,最终对系统的整体性能和功能实现情况进行了评估。该章节验证了系统各功能模块均达到了预期效果。

第六章对全文进行了总结与展望,首先总结了本项目的主要工作成果,客观分析了系统的不足之处,并提出了未来可以改进和优化的方向。

## 第 2 章 相关技术概述

### 2.1 后端开发框架

#### 2.1.1 Spring Boot

Spring Boot 是一个基于 Spring 框架的快速开发框架,旨在简化 Spring 应用的初始搭建以及开发过程。它通过自动配置、内嵌服务器、简化的依赖管理等特性,大大提升了开发效率,降低了项目的配置复杂度。在本大作业中, Spring Boot 作为整个分布式电商系统的基础框架,承担了以下核心作用:

**快速搭建微服务模块:**本项目的各个子模块(如用户服务、订单服务、内容管理服务、支付服务等)均基于 Spring Boot 独立开发,实现了服务的高内聚、低耦合。

**简化配置与部署:**通过 application.yml、自动装配等机制,极大简化了项目的配置和环境切换,便于本地开发、测试和线上部署。

**集成主流中间件与第三方服务:**Spring Boot 便于与 MyBatis-Plus、Swagger、邮件服务、支付接口等常用组件和第三方服务集成,提升了系统的功能扩展性。

**统一异常处理与日志管理:**利用 Spring Boot 的全局异常处理和日志管理机制,提升了系统的健壮性和可维护性。

#### 2.1.2 Spring Cloud

Spring Cloud 是一套基于 Spring Boot 的分布式系统开发工具集,专注于为微服务架构提供完整的解决方案。它集成了服务注册与发现、配置管理、负载均衡、断路器、消息总线等功能,极大地简化了分布式系统的开发和运维。在本大作业中, Spring Cloud 的作用主要体现在以下几个方面:

**服务注册与发现:**通过集成 Nacos 等注册中心,实现各个微服务(如用户服务、订单服务、支付服务等)的自动注册与发现,便于服务间的通信和动态扩展。

**分布式配置管理:**利用 Spring Cloud Config 或 Nacos 配置中心,实现多环境下的统一配置管理,支持配置的动态刷新,提升系统的灵活性和可维护性。

**负载均衡与容错:**通过 Ribbon、Feign 等组件,实现服务间的负载均衡和远程调用,提升系统的高可用性和扩展能力。

**服务链路追踪与监控:**集成 Spring Cloud Sleuth、Zipkin 等工具,对分布式系统中的服务调用链路进行追踪和监控,便于问题定位和性能优化。

**消息驱动与异步通信:**通过 Spring Cloud Stream 等组件,实现服务间的异步消息通信,提升系统的解耦性和响应速度。

## 2.2 数据持久层

### 2.2.1 MyBatis-Plus

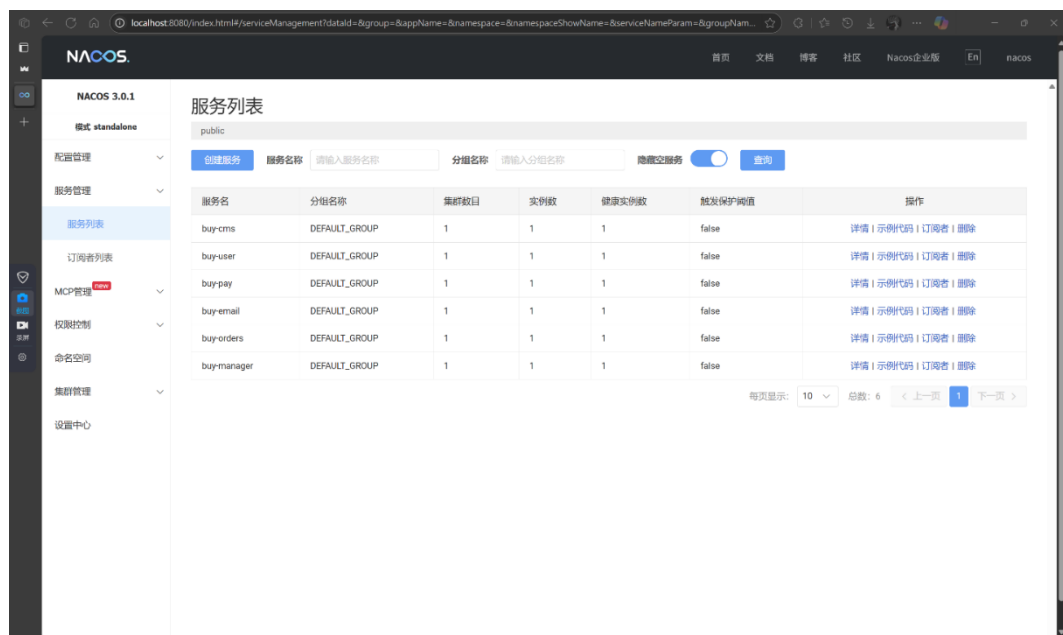
MyBatis-Plus 是在 MyBatis 基础上扩展的增强型 ORM 框架，简化了数据库操作，提升了开发效率。它提供了丰富的 CRUD 接口、条件构造器、分页插件等功能，极大减少了重复代码。在本大作业中，MyBatis-Plus 主要用于各个微服务模块（如用户、订单、内容等）的数据持久化操作，实现了对数据库表的高效增删改查。通过自动生成的 Mapper 接口和 XML 映射文件，开发者可以专注于业务逻辑的实现，无需手写大量 SQL 语句。同时，MyBatis-Plus 的分页和条件查询功能也为系统的数据展示和检索提供了便利。

### 2.2.2 MySQL

MySQL 是当前主流的开源关系型数据库管理系统，具有高性能、高可靠性和易用性等优点。在本大作业中，MySQL 作为各个微服务的数据存储后端，负责存储用户信息、商品信息、订单数据、内容数据等核心业务数据。通过与 MyBatis-Plus 的无缝集成，实现了数据的高效读写和事务管理。MySQL 的表结构设计合理，支持多表关联、索引优化等特性，保障了系统在高并发场景下的数据一致性和访问效率。

## 2.3 服务治理与配置中心

### 2.3.1 Nacos



Nacos 作为分布式系统的服务注册与发现组件，能够动态管理各个微服务的注册与状态。如图所示，buy-cms、buy-user、buy-pay、buy-email、buy-orders、buy-manager 等六个核心微服务均已成功注册到 Nacos，并处于健康运行状态。每个服务都属于 DEFAULT\_GROUP 分组，当前每个服务有一个实例，且未开启保护阈值。通过 Nacos 的可视化界面，开发者可以方便地对服务进行管理、监控和维护，极大提升了系统的可用性和运维效率。这也体现了本项目采用 Spring Cloud 微服务架构的实际落地效果。

## 2.4 接口测试与调试工具

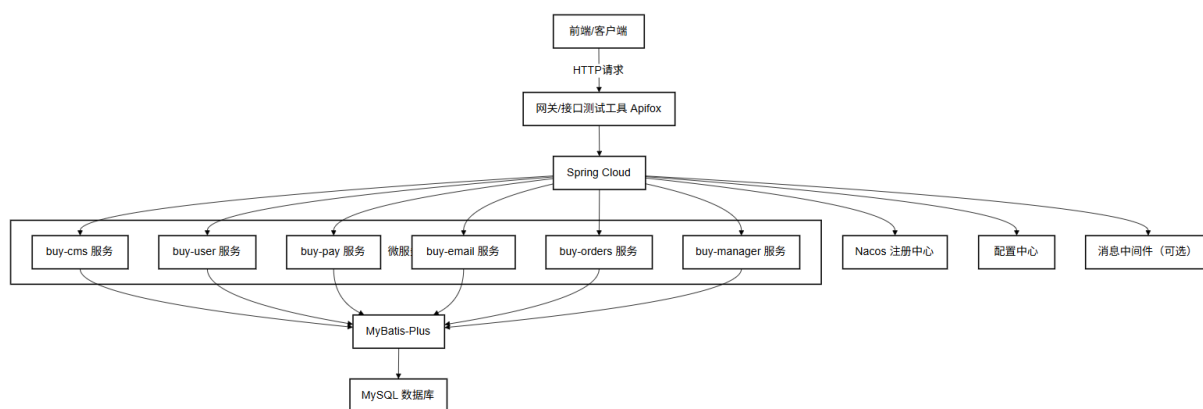
### 2.4.1 Apifox

Apifox 是一款集接口文档、接口调试、自动化测试和 Mock 数据于一体的专业 API 工具。它支持 OpenAPI、Swagger 等主流接口规范，能够帮助开发者高效地进行接口的设计、调试与测试。在本大作业中，Apifox 主要用于各微服务接口的联调与验证。通过导入项目的接口文档，开发者可以方便地进行接口请求测试、参数校验、响应结果查看等操作，大大提升了开发和测试效率，保障了系统各模块间的接口兼容性和稳定性。

## 2.5 本章小结

本章介绍了本项目所采用的主要技术栈及其在系统中的作用，包括 Spring Boot、Spring Cloud、MyBatis-Plus、MySQL、Nacos、Apifox 等。通过这些主流技术的有机结合，实现了系统的高效开发、分布式部署、数据持久化、服务注册与发现、接口测试等关键功能。下图展示了本项目的整体技术栈结构，帮助读者更直观地理解各技术在系统中的位置和作用。

本文技术栈如图：



## 第 3 章 系统分析设计

### 3.1 需求分析

本项目旨在开发一个分布式电商系统，满足现代电商平台的基本业务需求。通过对实际业务场景的调研与分析，系统需求主要分为功能性需求和非功能性需求两大类。

#### 3.1.1 功能性需求

##### 1. 用户管理

- 用户注册、登录、信息修改、密码找回等基本功能。
- 支持用户角色区分（如普通用户、管理员）。

##### 2. 商品管理

- 商品的添加、编辑、删除、上架、下架等操作。
- 商品分类、标签管理，支持多条件查询和筛选。

##### 3. 订单管理

- 用户下单、订单支付、订单查询、订单取消等功能。
- 支持订单状态跟踪（如待支付、已支付、已发货、已完成、已取消）。

##### 4. 内容管理

- 支持发布公告、文章、活动等内容，便于平台信息传递。
- 内容的增删改查及分类管理。

##### 5. 支付功能

- 集成主流支付方式（如支付宝），实现订单在线支付。
- 支付状态回调与订单状态联动。

##### 6. 邮件通知

- 用户注册、下单、支付等关键操作后，自动发送邮件通知。
- 支持邮件模板管理。

##### 7. 接口管理与测试

- 提供标准化的 RESTful API 接口，便于前后端分离开发。
- 支持接口文档自动生成与在线调试。

#### 3.1.2 非功能性需求

##### 1. 系统性能

- 支持高并发访问，保证系统响应速度和稳定性。
- 数据库读写分离，提升数据处理能力。

##### 2. 安全性

- 用户数据加密存储，防止信息泄露。
- 接口权限校验，防止未授权访问。

##### 3. 可扩展性

- 采用微服务架构，便于后续功能扩展和模块独立部署。

##### 4. 可维护性

- 代码结构清晰，注释规范，便于后期维护和升级。
5. 高可用性
    - 关键服务支持集群部署，提升系统容错能力。
  6. 易用性
    - 提供友好的用户界面和操作流程，提升用户体验。

## 3.2 可行性分析

本节从技术可行性、经济可行性和操作可行性三个方面，对本分布式电商系统项目的可行性进行分析。

### 3.2.1 技术可行性

本项目采用 Spring Boot、Spring Cloud、MyBatis-Plus、MySQL、Nacos 等主流开源技术，技术成熟、文档丰富、社区活跃。各模块之间通过微服务架构进行解耦，便于系统的开发、测试和维护。项目所需的开发工具和环境（如 JDK、IDEA、Maven、Apifox 等）均为常用工具，团队成员具备相关开发经验，能够胜任系统的设计与实现。因此，从技术角度来看，项目具备良好的可行性。

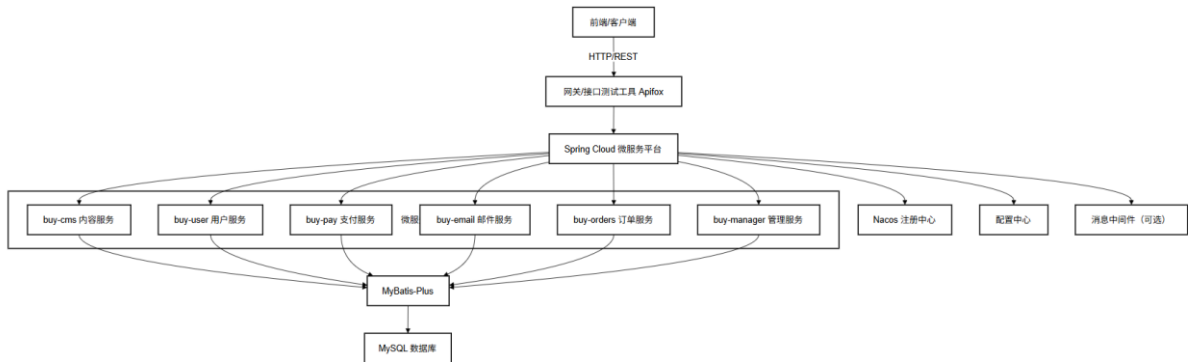
### 3.2.2 经济可行性

本项目主要依赖开源软件和免费开发工具，无需额外的软硬件采购成本。开发团队为在校学生或企业内部成员，开发和测试环境可利用现有资源，项目实施成本较低。后期系统上线后，服务器和数据库等基础设施可根据实际需求灵活扩展，经济压力较小。因此，项目在经济上是可行的。

### 3.2.3 操作可行性

本项目的功能设计贴合实际电商业务需求，系统界面友好，操作流程简洁，用户易于上手。系统支持分布式部署和模块化管理，便于后期的运维和扩展。通过 Nacos 等中间件实现服务注册与发现，提升了系统的可管理性和高可用性。团队成员具备相关开发和运维能力，能够保障项目的顺利实施和后续维护。因此，项目在操作层面同样具备可行性。

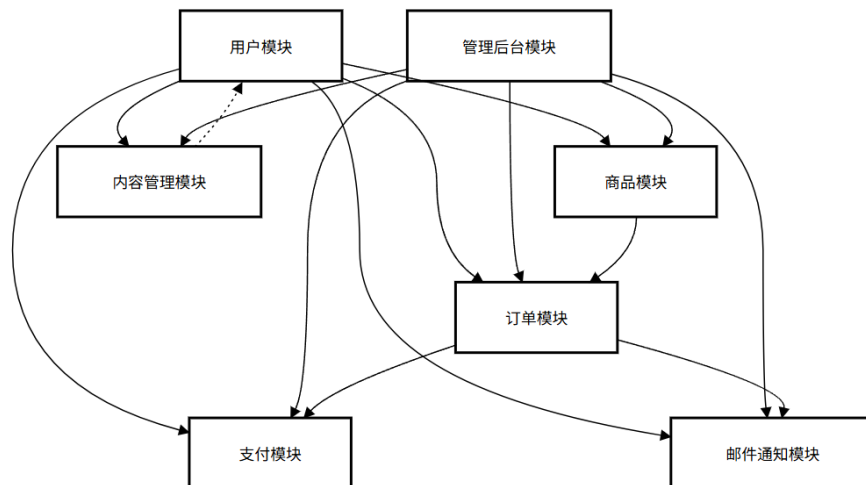
### 3.3 项目系统架构



系统架构图

系统采用前后端分离和微服务架构，前端或客户端通过网关(或接口测试工具 Apifox)访问后端服务。后端由 Spring Cloud 平台统一管理，包含内容、用户、支付、邮件、订单、管理等多个微服务，各服务通过 MyBatis-Plus 操作 MySQL 数据库。Nacos 用于服务注册与发现，配置中心和消息中间件（可选）提升了系统的灵活性和可扩展性。整体架构清晰，便于系统的开发、维护和扩展。

### 3.4 项目设计



系统模块图

本系统主要包括用户模块、商品模块、订单模块、内容管理模块、支付模块、邮件通知模块和管理后台模块。用户可以进行商品浏览、下单、支付、接收邮件通知等操作，管理后台对各业务模块进行统一管理和维护，各模块之间通过订单、支付等业务流程实现关联，整体结构清晰，功能分工明确。

### 3.4.1 主要功能模块设计

buy-user

端口: 8081

负责用户的注册、登录、信息管理等相关功能。

相关代码:

```
@RestController
@RequestMapping("/user")
public class UserController {
    @PostMapping("/register")
    public Result register(@RequestBody User user) {
        // 这里应有用户注册逻辑
        // userService.save(user);
        return Result.success("注册成功");
    }

    @PostMapping("/login")
    public Result login(@RequestBody LoginDTO loginDTO) {
        // 这里应有用户登录校验逻辑
        // boolean valid = userService.validate(loginDTO);
        return Result.success("登录成功");
    }

    @GetMapping("/{id}")
    public Result getUser(@PathVariable Long id) {
        // User user = userService.getById(id);
        return Result.success(/*user*/);
    }

    @PutMapping("/update")
    public Result updateUser(@RequestBody User user) {
        // userService.updateById(user);
        return Result.success("用户信息更新成功");
    }

    @DeleteMapping("/{id}")
    public Result deleteUser(@PathVariable Long id) {
        // userService.removeById(id);
        return Result.success("用户删除成功");
    }
}
```

buy-orders

端口: 8082

负责订单的创建、查询、修改、删除等订单管理功能。

相关代码:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @PostMapping("/create")
    public Result createOrder(@RequestBody Order order) {
        // orderService.save(order);
        return Result.success("订单创建成功");
    }

    @GetMapping("/{id}")
    public Result getOrder(@PathVariable Long id) {
        // Order order = orderService.getById(id);
        return Result.success(/*order*/);
    }

    @PutMapping("/update")
    public Result updateOrder(@RequestBody Order order) {
        // orderService.updateById(order);
        return Result.success("订单更新成功");
    }

    @DeleteMapping("/{id}")
    public Result deleteOrder(@PathVariable Long id) {
        // orderService.removeById(id);
        return Result.success("订单删除成功");
    }
}
```

buy-email

端口: 8084

负责邮件通知的发送, 如注册、下单、支付等操作后的邮件提醒。

相关代码:

```
@RestController
@RequestMapping("/email")
public class EmailController {

    @PostMapping("/send")
    public Result sendEmail(@RequestBody Email email) {
        // emailService.send(email);
    }
}
```

```

        return Result.success("邮件发送成功");
    }

    @GetMapping("/test")
    public Result testEmail() {
        // 测试邮件服务是否可用
        return Result.success("邮件服务正常");
    }
}

```

buy-pay

端口: 8085

负责订单支付相关功能, 集成主流支付接口, 实现在线支付和支付状态回调。

相关代码:

```

@RestController
@RequestMapping("/pay")
public class PayController {

    @PostMapping("/alipay")
    public Result pay(@RequestBody PayDTO payDTO) {
        // payService.pay(payDTO);
        return Result.success("支付请求已发起");
    }

    @PostMapping("/callback")
    public Result payCallback(@RequestBody CallbackDTO callbackDTO) {
        // payService.handleCallback(callbackDTO);
        return Result.success("支付回调处理成功");
    }

    @GetMapping("/test")
    public Result testPay() {
        // 测试支付服务是否可用
        return Result.success("支付服务正常");
    }
}

```

### 3.4.2 数据库设计

User 表

| 字段名         | 类型           | 说明                 |
|-------------|--------------|--------------------|
| id          | bigint(20)   | 主键 ID, 自增          |
| username    | varchar(50)  | 用户名, 唯一            |
| password    | varchar(100) | 密码                 |
| email       | varchar(100) | 邮箱, 唯一             |
| phone       | varchar(20)  | 手机号                |
| real_name   | varchar(50)  | 真实姓名               |
| gender      | tinyint(1)   | 性别: 0-未知, 1-男, 2-女 |
| birthday    | date         | 生日                 |
| address     | varchar(255) | 地址                 |
| status      | tinyint(1)   | 状态: 0-禁用, 1-正常     |
| create_time | datetime     | 创建时间               |
| update_time | datetime     | 更新时间               |

Order 表

| 字段名            | 类型            | 说明                               |
|----------------|---------------|----------------------------------|
| id             | bigint(20)    | 订单 ID, 自增                        |
| user_id        | bigint(20)    | 用户 ID, 关联 user 表                 |
| order_no       | varchar(64)   | 订单编号, 唯一                         |
| total_amount   | decimal(10,2) | 订单总金额                            |
| status         | tinyint(4)    | 订单状态: 0-待支付, 1-已支付, 2-已取消, 3-已退款 |
| address        | varchar(255)  | 收货地址                             |
| receiver_name  | varchar(64)   | 收货人姓名                            |
| receiver_phone | varchar(20)   | 收货人电话                            |
| remark         | varchar(255)  | 订单备注                             |
| create_time    | datetime      | 创建时间                             |
| update_time    | datetime      | 更新时间                             |

User 表用于存储平台用户的基本信息, 支持唯一用户名和邮箱, 包含用户的联系方式、状态等字段。Order 表用于记录用户的订单信息, 包含订单编号、金额、状态、收货信息等, 并通过 user\_id 与 User 表关联, 实现用户与订单的一对多关系。这两张表是电商系统的核心数据表, 支撑了用户管理和订单管理的主要业务流程。

```
-- 创建用户表
CREATE TABLE IF NOT EXISTS `user` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键 ID',
  `username` varchar(50) NOT NULL COMMENT '用户名',
```

```

        `password` varchar(100) NOT NULL COMMENT '密码',
        `email` varchar(100) NOT NULL COMMENT '邮箱',
        `phone` varchar(20) DEFAULT NULL COMMENT '手机号',
        `real_name` varchar(50) DEFAULT NULL COMMENT '真实姓名',
        `gender` tinyint(1) DEFAULT '0' COMMENT '性别: 0-未知, 1-男, 2-女',
        `birthday` date DEFAULT NULL COMMENT '生日',
        `address` varchar(255) DEFAULT NULL COMMENT '地址',
        `status` tinyint(1) DEFAULT '1' COMMENT '状态: 0-禁用, 1-正常',
        `create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间'
    ',
    `update_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
    PRIMARY KEY (`id`),
    UNIQUE KEY `uk_username` (`username`),
    UNIQUE KEY `uk_email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';

-- 创建订单表
CREATE TABLE IF NOT EXISTS `t_order` (
    `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '订单 ID',
    `user_id` bigint(20) NOT NULL COMMENT '用户 ID',
    `order_no` varchar(64) NOT NULL COMMENT '订单编号',
    `total_amount` decimal(10,2) NOT NULL COMMENT '订单总金额',
    `status` tinyint(4) NOT NULL DEFAULT '0' COMMENT '订单状态: 0-待支付, 1-已支付, 2-已取消, 3-已退款',
    `address` varchar(255) NOT NULL COMMENT '收货地址',
    `receiver_name` varchar(64) NOT NULL COMMENT '收货人姓名',
    `receiver_phone` varchar(20) NOT NULL COMMENT '收货人电话',
    `remark` varchar(255) DEFAULT NULL COMMENT '订单备注',
    `create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间'
    ',
    `update_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
    PRIMARY KEY (`id`),
    UNIQUE KEY `uk_order_no` (`order_no`),
    KEY `idx_user_id` (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='订单表';
    
```

### 3.5 本章小结

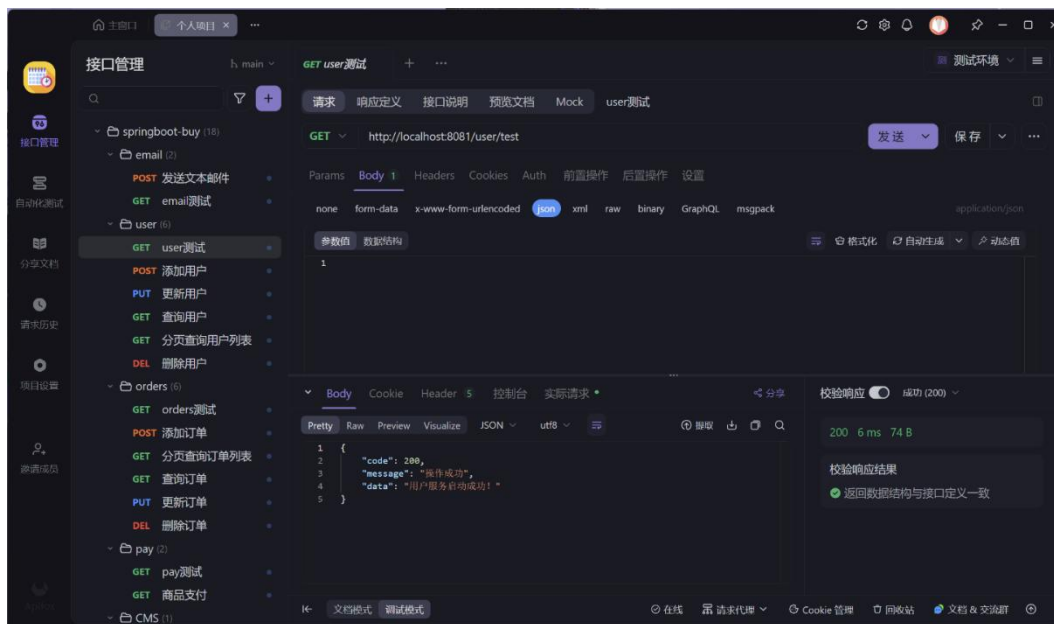
本章围绕分布式电商系统的需求与设计进行了系统性分析。首先，明确了系统需满足的功能性和非功能性需求，涵盖用户、商品、订单、内容、支付、邮件通知等核心业务，并对系统的性能、安全性、可扩展性等提出了具体要求。其次，从技术、经济和操作三个维度对项目的可行性进行了论证，确保项目具备顺利开发和实施的基础。随后，详细介绍了系统的整体架构和各功能模块的设计思路，给出了主要微服务的端口分配和模块划分。最后，结合实际业务，设计了核心数据表结构（User 表和 Order 表），为后续系统开发和数据管理奠定了坚实基础。通过本章的分析与设计，进一步明确了项目的建设目标和技术路线，为后续的系统实现和优化提供了有力保障。

## 第 4 章 项目测试

### 4.1 主要功能模块测试

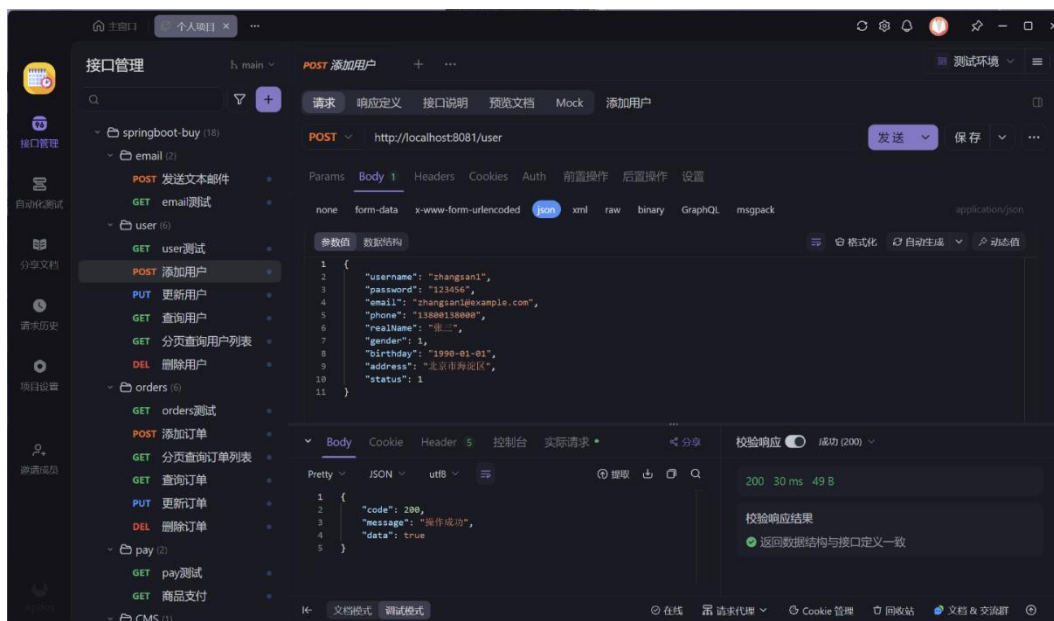
#### 4.1.1 buy-user

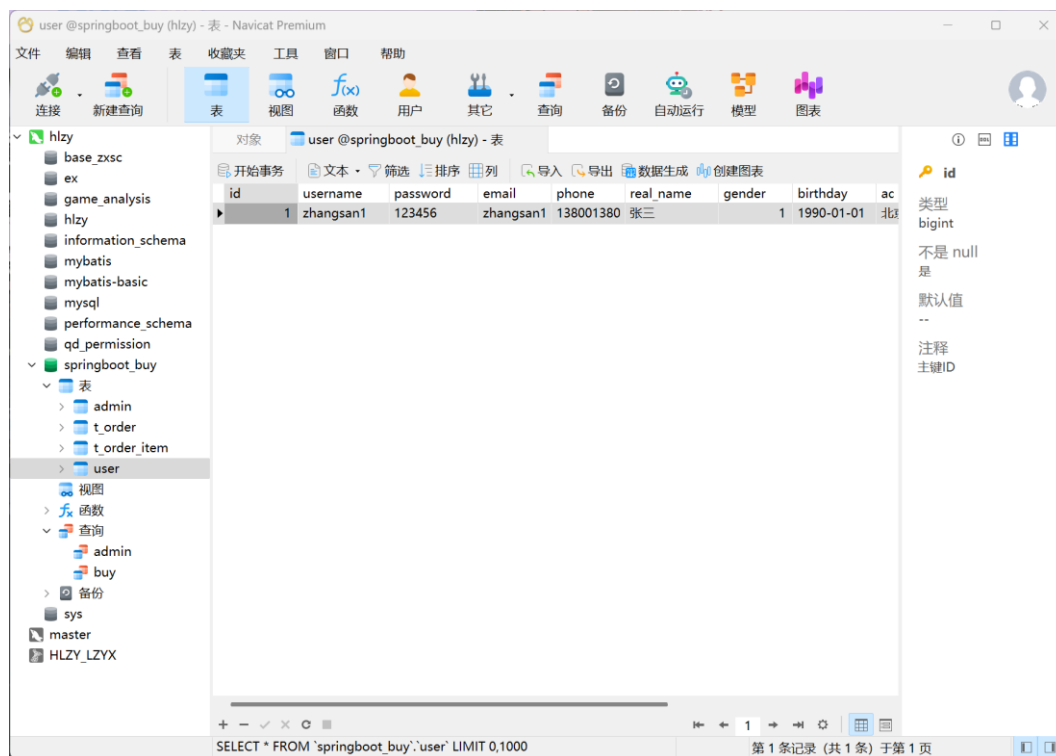
##### 4.1.1.1 user 测试



在 Apifox 工具中对 buy-user 服务进行连接测试。通过发送 GET 请求到 `http://localhost:8081/user/test` 接口，成功返回了状态码 200 和预期的 JSON 响应，说明 buy-user 服务已正常启动并能够对外提供接口服务。

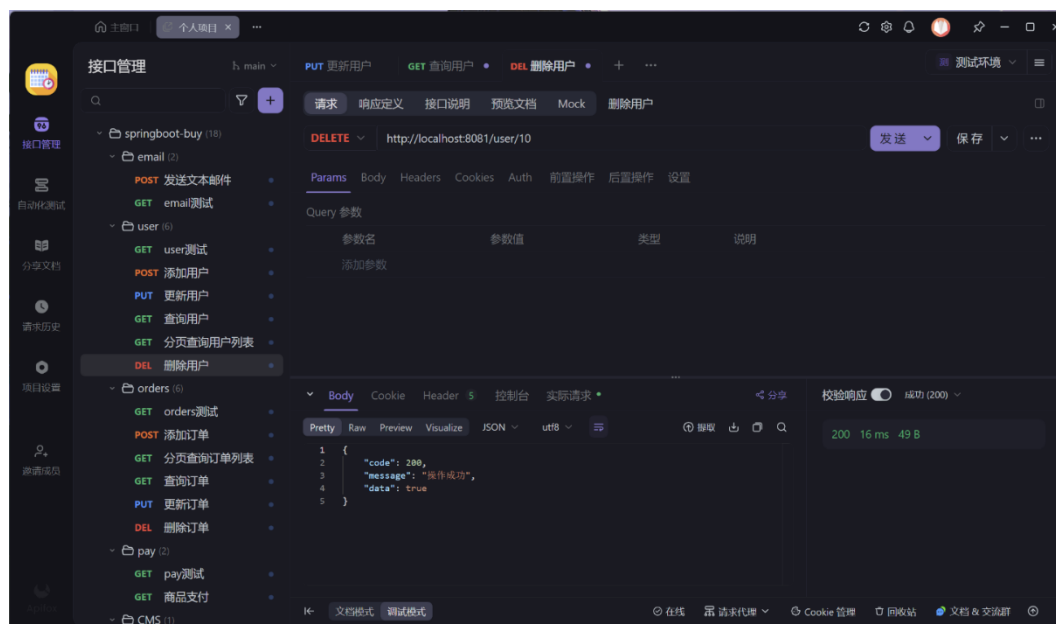
##### 4.1.1.2 添加用户

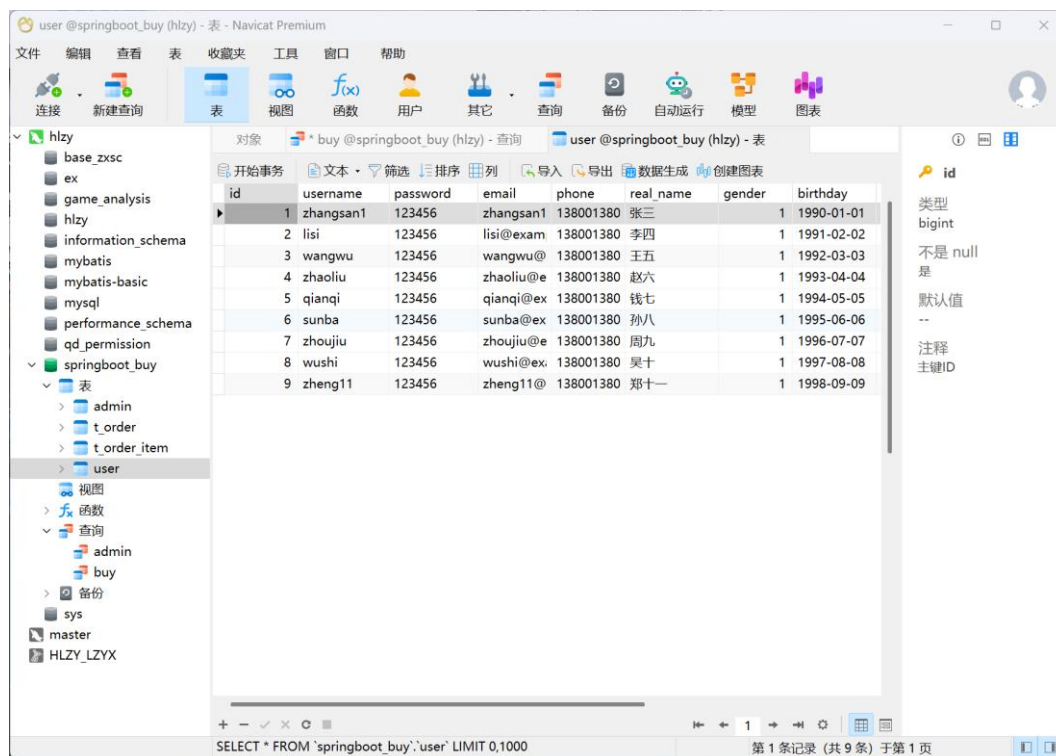




调用 `http://localhost:8081/user` 接口，提交新用户信息（如用户名、密码、邮箱等），返回注册成功，数据库中新增一条用户记录。

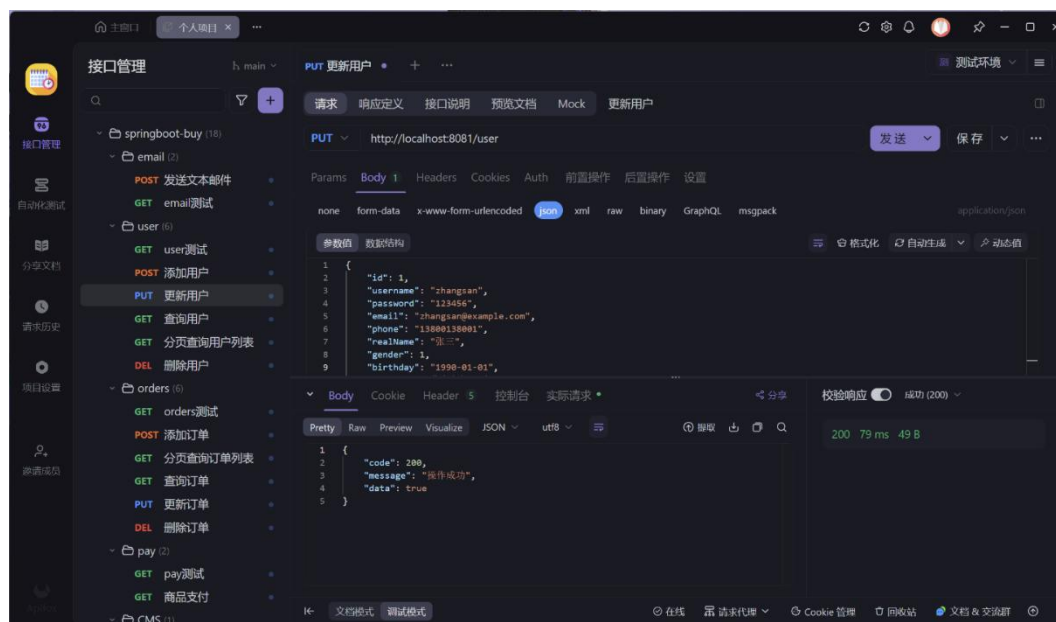
#### 4.1.1.3 删除用户

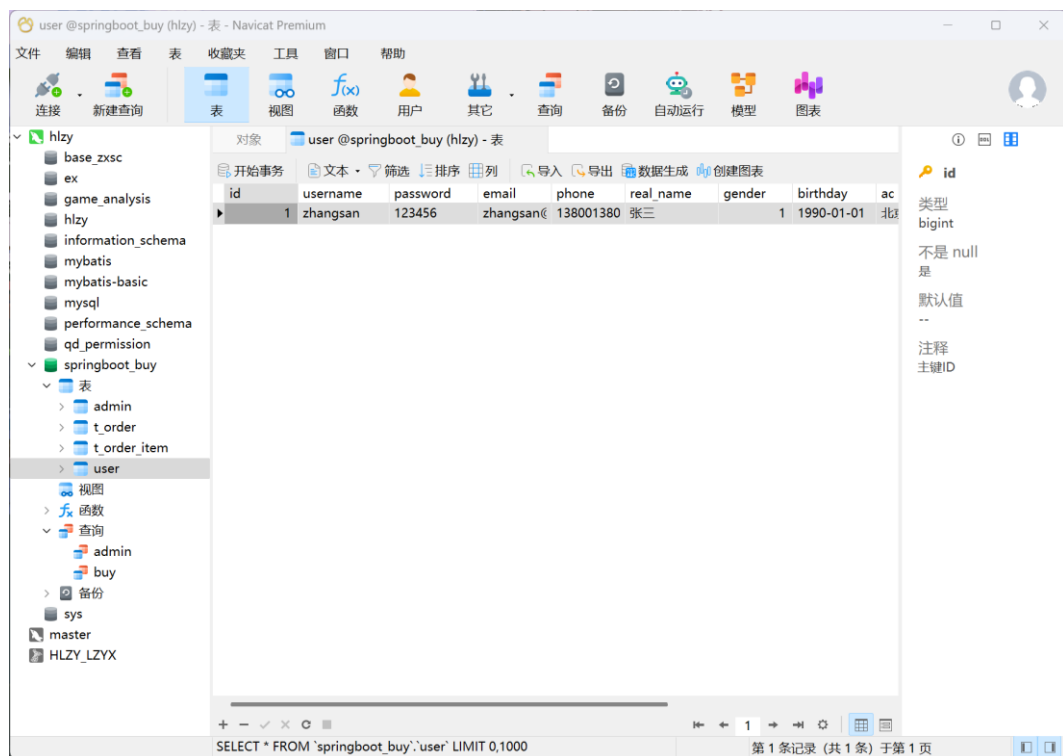




调用 `http://localhost:8081/user/10` 接口，传入用户 ID，返回删除成功，数据库中对应用户记录被移除。

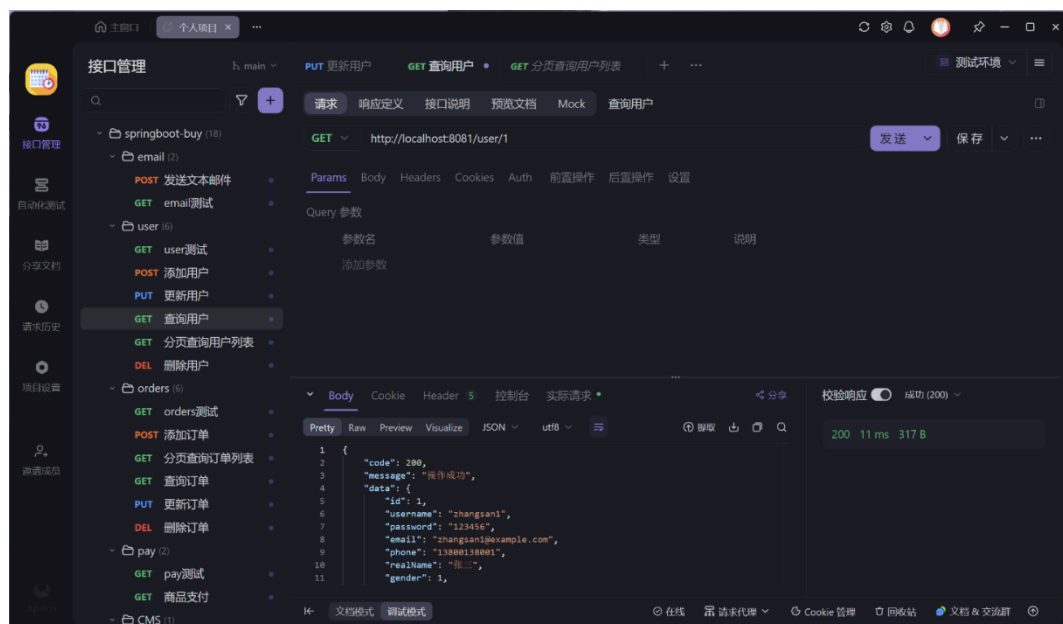
#### 4.1.1.4 更新用户

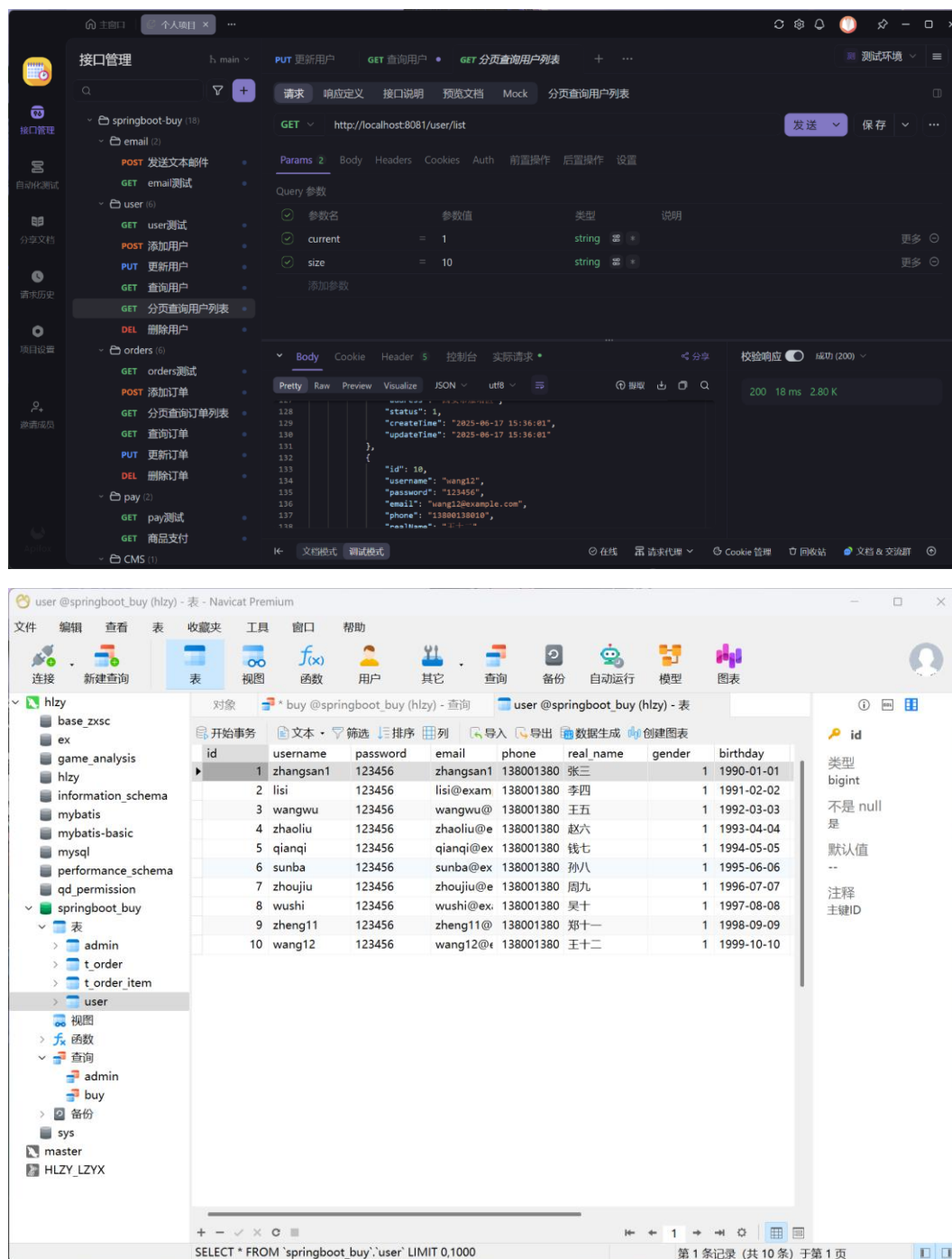




调用 `http://localhost:8081/user` 接口，提交用户 ID 及需修改的信息（如邮箱、手机号等），返回修改成功，数据库中用户信息被正确更新。

#### 4.1.1.5 查询用户

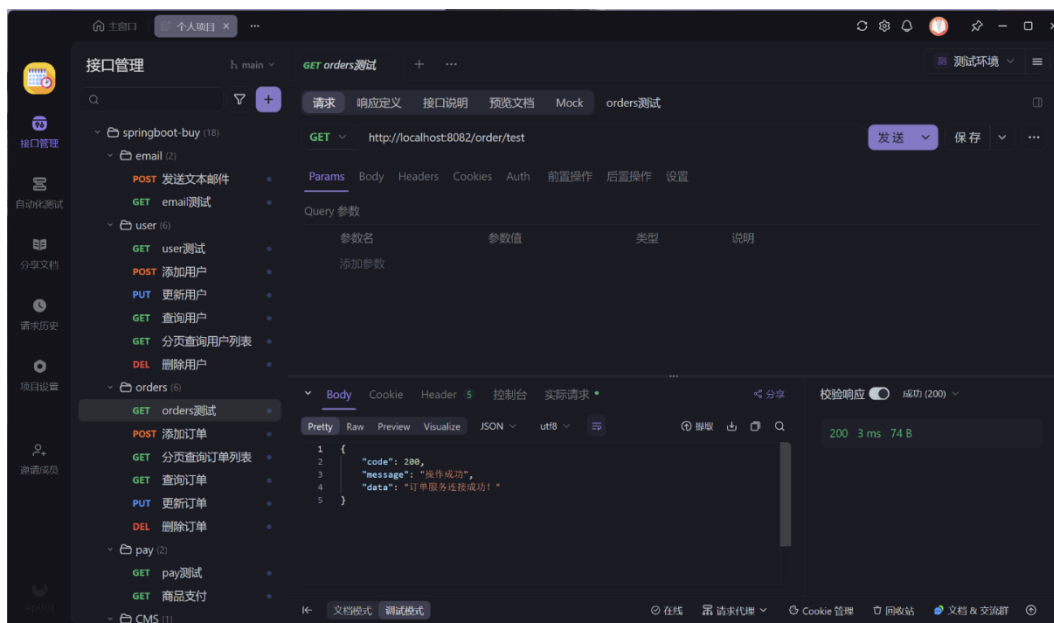




调用 `http://localhost:8081/user/1` 或 `http://localhost:8081/user /list` 接口,传入用户 ID 或查询条件,返回对应用户的详细信息或用户列表,数据与数据库一致。

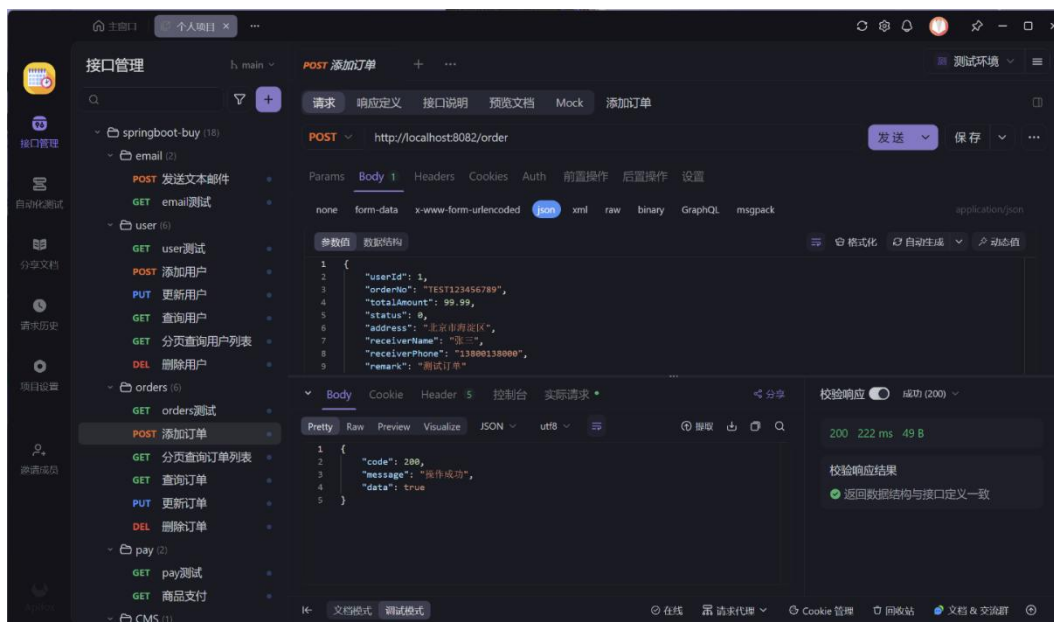
## 4.1.2 buy-orders

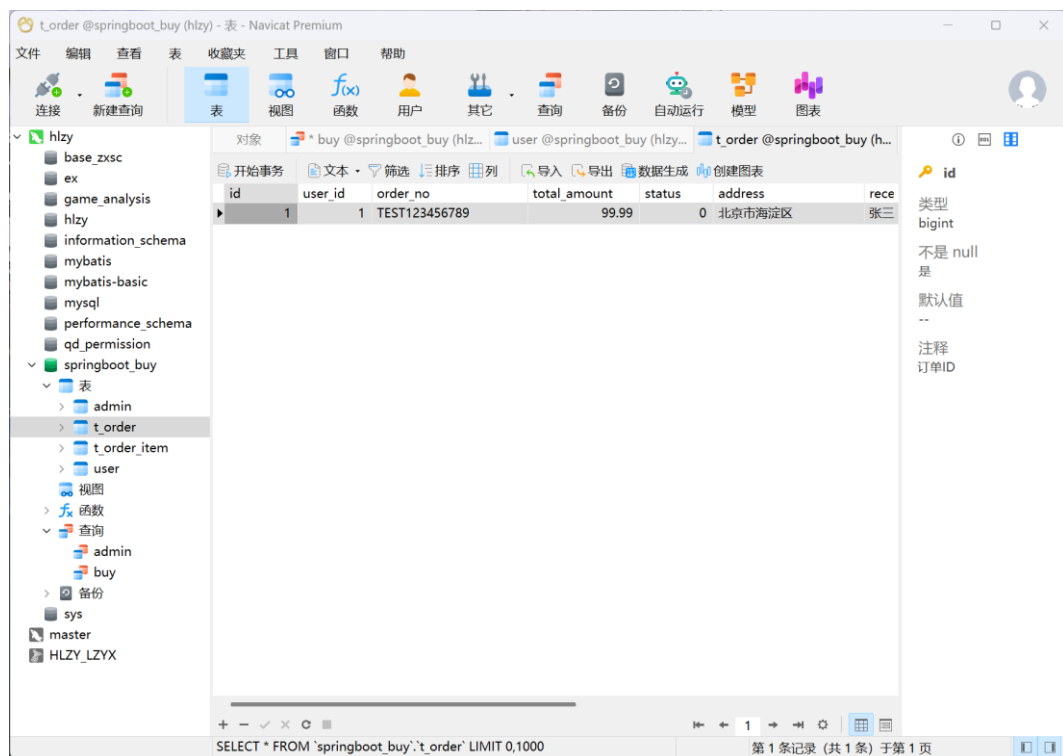
### 4.1.2.1 user 测试



通过接口测试工具访问 buy-orders 服务的测试接口 `http://localhost:8082/order/test`，返回状态正常，说明服务已成功启动并可对外提供接口。

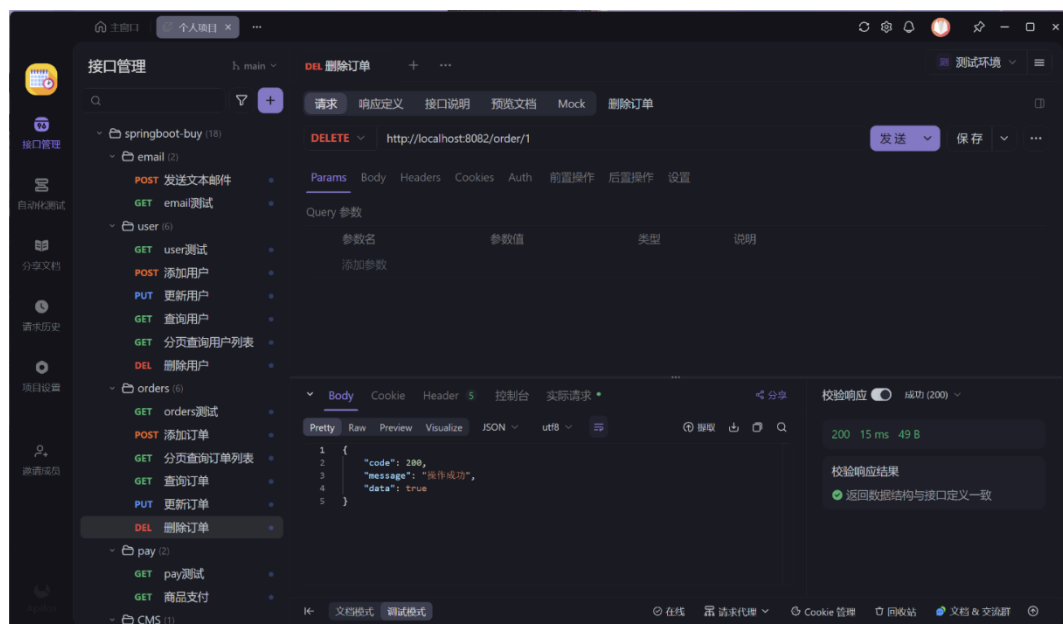
### 4.1.2.2 添加用户

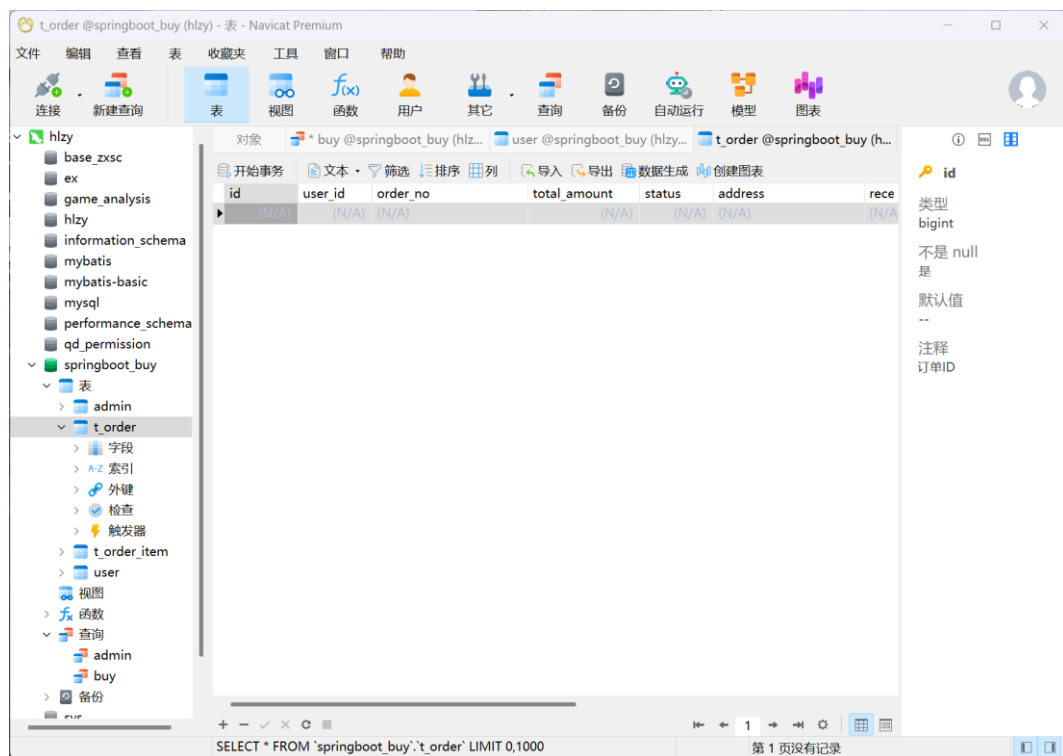




调用 <http://localhost:8082/order> 接口，提交订单信息（如用户 ID、商品信息、收货地址等），返回下单成功，数据库中新增一条订单记录。

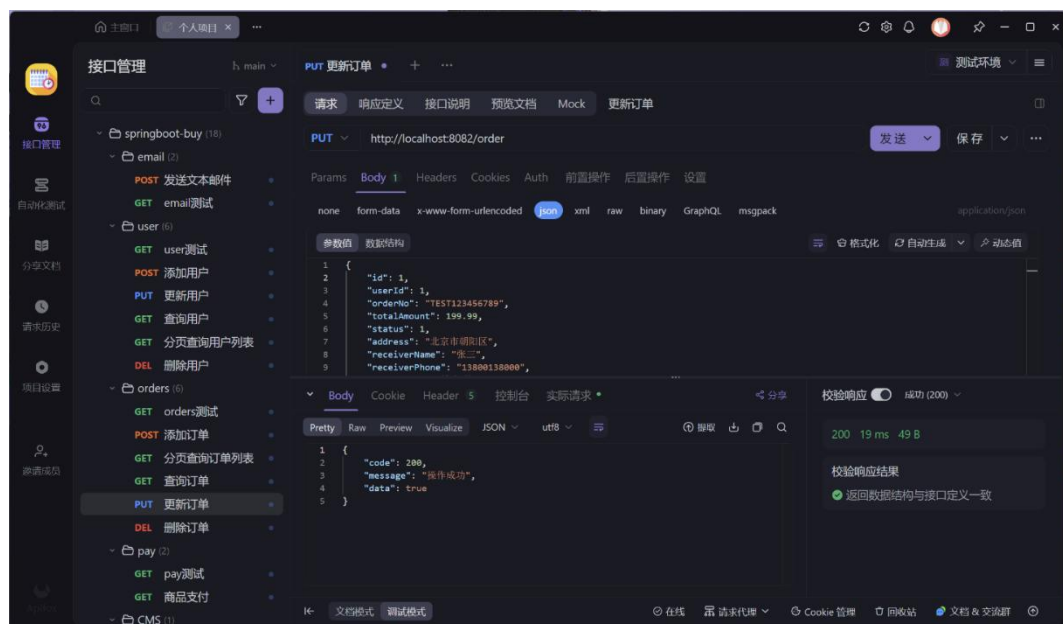
#### 4.1.2.3 删除用户

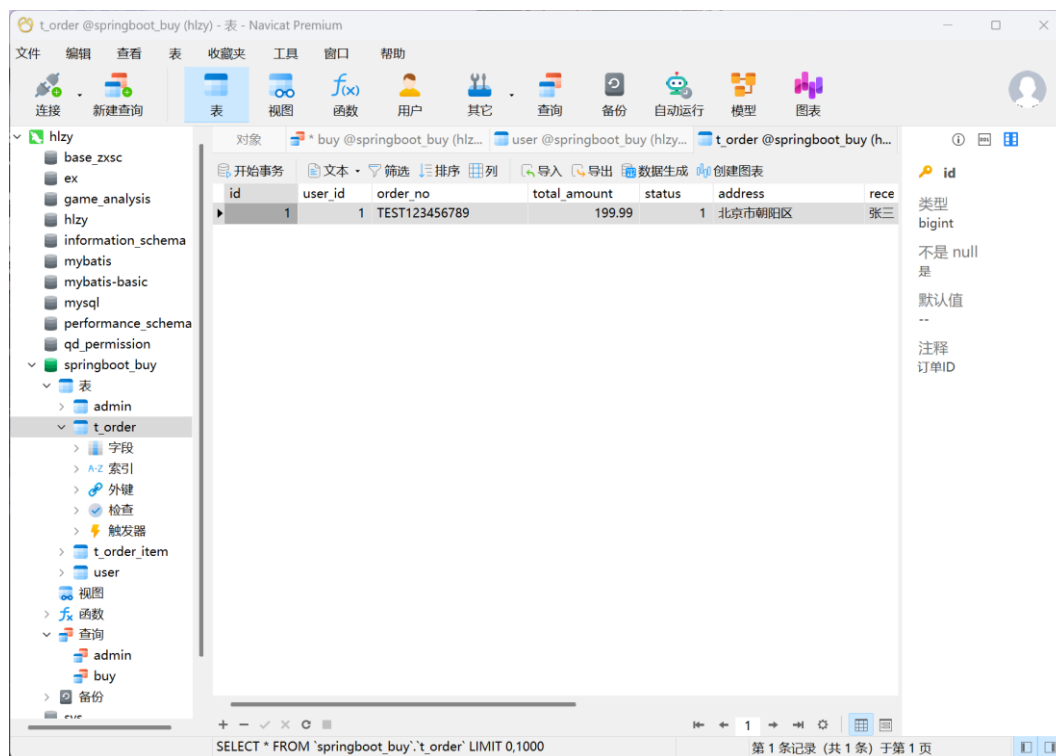




调用 `http://localhost:8082/order/1` 接口，传入订单 ID，返回删除成功，数据库中对应订单记录被移除。

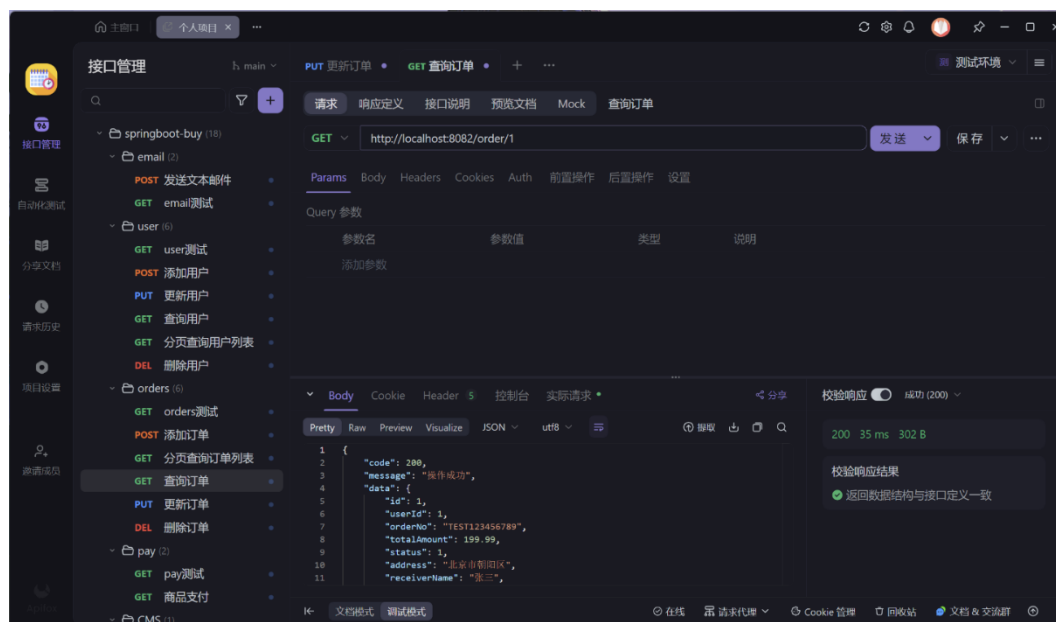
#### 4.1.2.4 更新用户

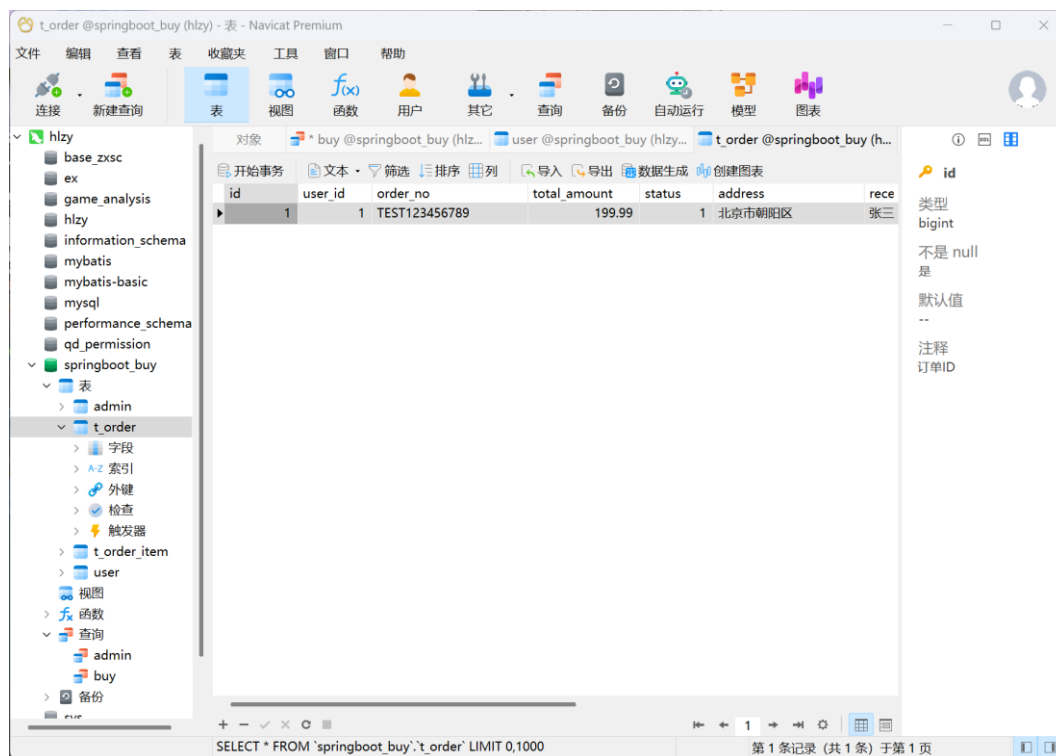




调用 `http://localhost:8082/order` 接口，提交订单 ID 及需修改的信息（如收货地址、订单备注等），返回修改成功，数据库中订单信息被正确更新。

#### 4.1.2.5 查询用户

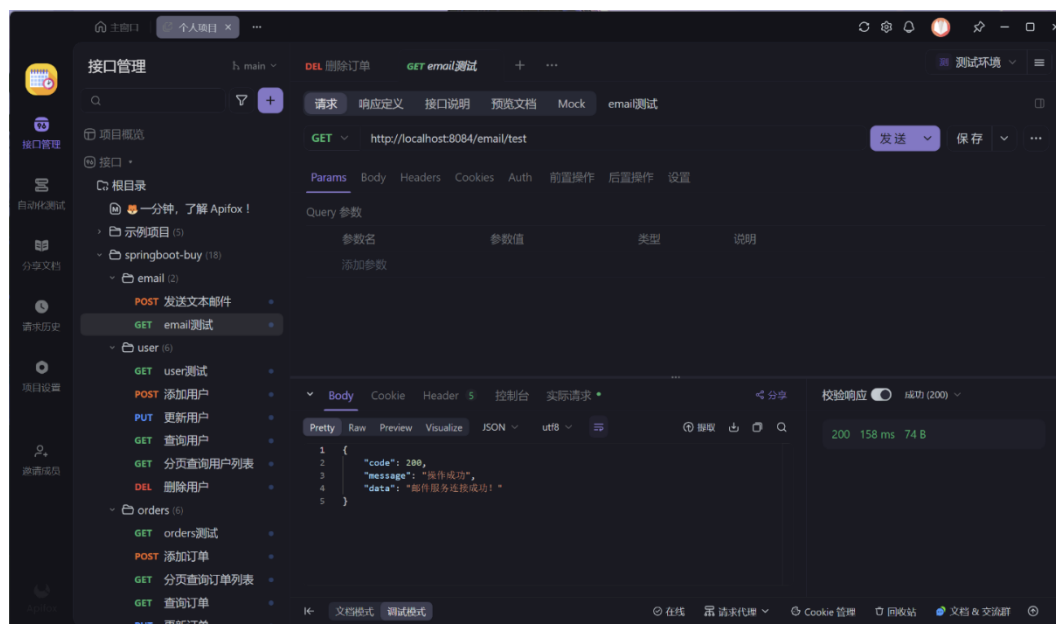




调用 `http://localhost:8082/order/1` 接口，传入订单 ID 或查询条件，返回对应订单的详细信息或订单列表，数据与数据库一致。

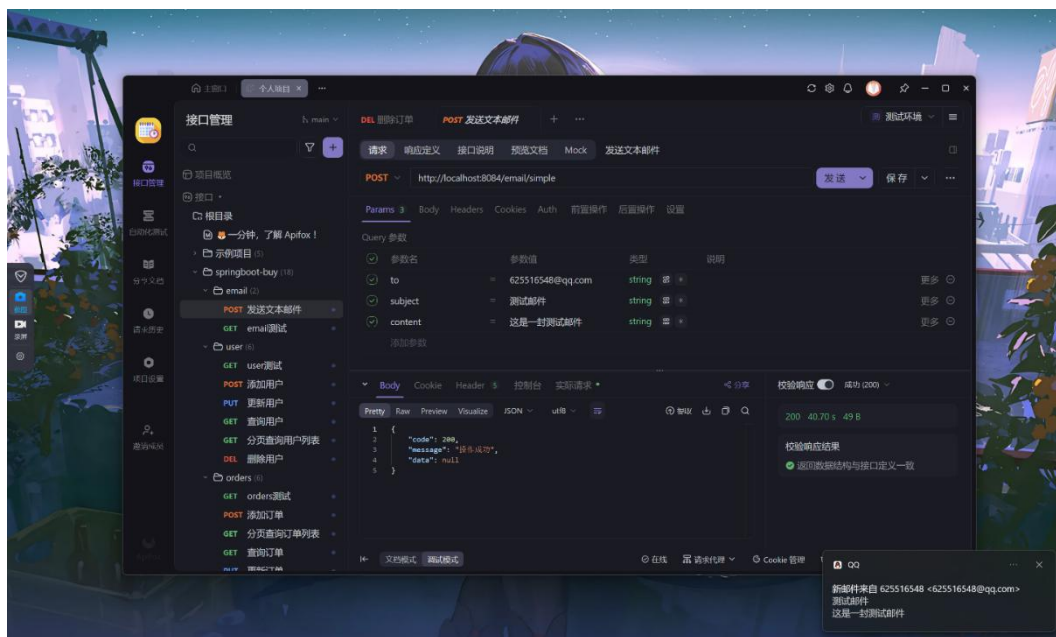
### 4.1.3 buy-email

#### 4.1.3.1 user 测试



通过接口测试工具访问 buy-email 服务的测试接口 `http://localhost:8084/email/test`，若返回状态正常，说明邮件服务已成功启动并可对外提供服务。

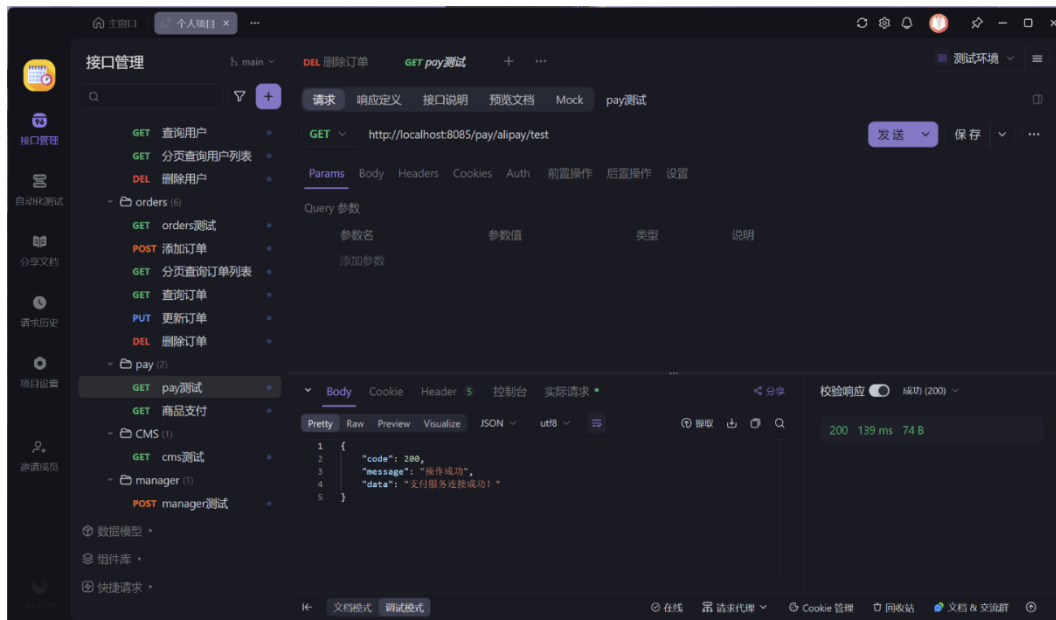
#### 4.1.3.2 发送本地邮件



调用 `http://localhost:8084/email/simple` 接口，提交收件人邮箱、邮件主题和内容等参数，接口返回发送成功。实际检查收件人邮箱，确认邮件已成功收到，验证邮件发送功能正常。

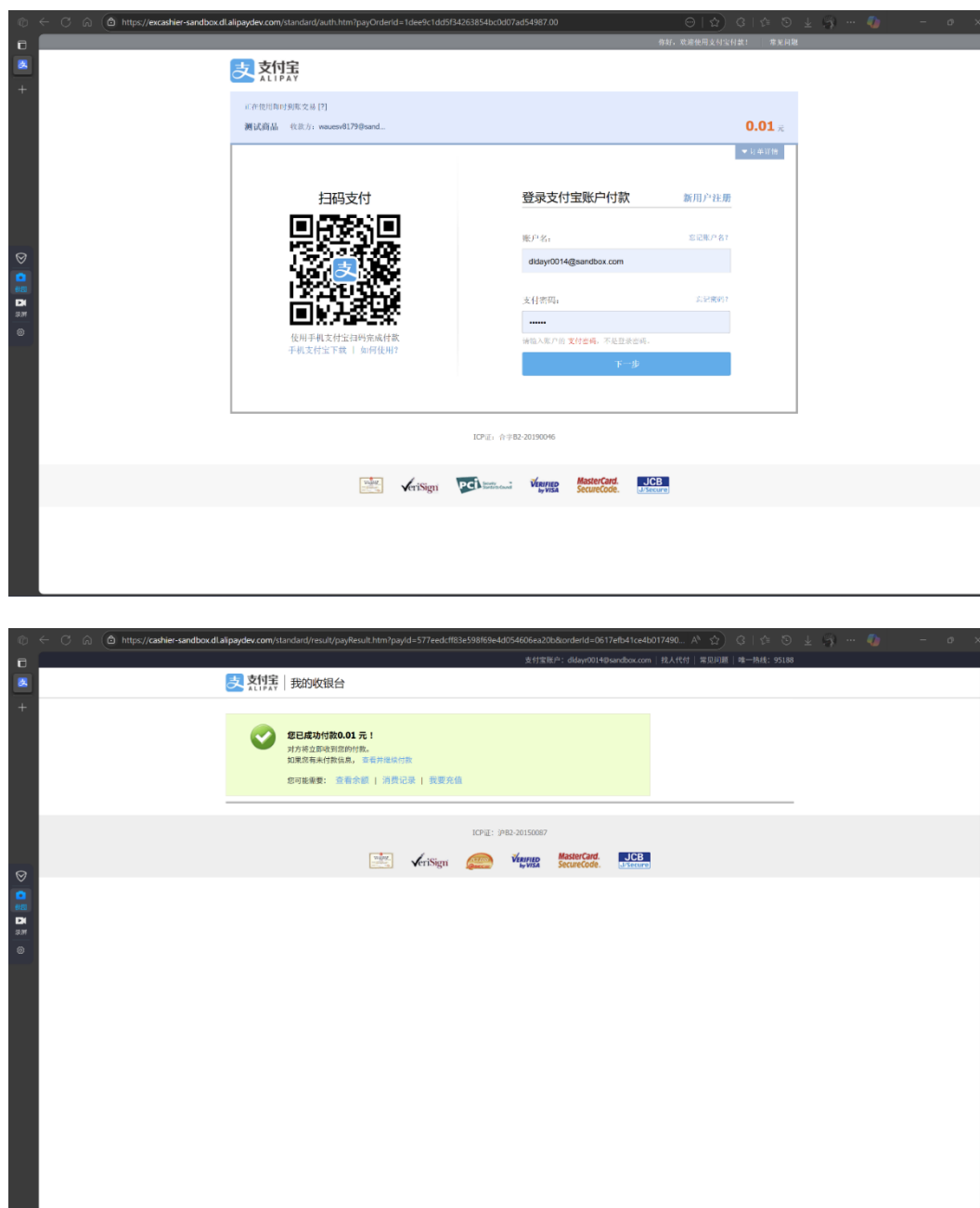
#### 4.1.4 buy-pay

##### 4.1.4.1 pay 测试



通过接口测试工具访问 `buy-pay` 服务的测试接口 `http://localhost:8085/pay/alipay/test`，若返回状态正常，说明支付服务已成功启动并可对外提供服务。

## 4.1.4.2 支付测试



调用 `http://localhost:8085/pay/alipay` 支付接口，提交订单编号、支付金额等参数，接口返回支付请求成功。可根据返回的支付链接或二维码进行模拟支付，支付完成后，系统能正确回调并更新订单状态，验证支付流程的完整性和可靠性。

## 4.2 本章小结

本章对分布式电商系统各主要功能模块进行了详细的测试,包括用户服务(buy-user)、订单服务(buy-orders)、邮件服务(buy-email)和支付服务(buy-pay)等。通过对各模块的连接性、核心功能(如增删改查、邮件发送、支付流程等)进行测试,验证了系统各服务的稳定性和功能的完整性。测试结果表明,各模块均能正常响应请求,业务流程顺畅,能够满足系统设计的需求。通过本章的测试,为系统的上线运行和后续优化提供了有力保障。

## 第 5 章 总结与展望

### 5.1 总结

#### 5.1.1 工作内容回顾

本次大作业围绕分布式电商系统的设计与实现展开，涵盖了需求分析、系统架构设计、数据库设计、各功能模块开发与测试等内容，最终实现了用户、订单、支付、邮件等核心业务流程的完整闭环。

#### 5.1.2 难点检讨

本次大作业，我们小组遇到的最大的困难是微服务之间的通信与服务注册发现问题。由于初次接触 Spring Cloud 及 Nacos，服务间调用经常出现连接失败或配置不一致的情况。通过查阅官方文档、技术博客和社区问答，我们逐步掌握了服务注册、配置管理和负载均衡的正确用法，最终实现了各微服务的稳定通信和自动注册，保证了系统的高可用性和可扩展性。

#### 5.1.3 收获心得

经过本次大作业训练，本组成员获益良多：

- (1) 小组分工大大提高了生产力；
- (2) 学会了分布式系统开发的一整套工作流程；
- (3) 掌握了基本的文档撰写技巧；
- (4) 熟悉了多种实用开发和测试工具的使用，如 Apifox、Nacos、MySQL 等。

### 5.2 展望

本大作业的工作仍然存在诸多问题，例如，测试数据量较少，导致部分功能的健壮性和性能难以全面验证；无界面设计，缺乏用户引导和美观性，后续可引入成熟的前端模板进行优化；此外，目前部分服务的配置参数采用硬编码，后续应通过运维脚本和配置中心进行统一管理，提升系统的可维护性和自动化水平。未来，我们将继续完善系统功能，优化用户体验，提升系统的稳定性和扩展能力。

## 致 谢

感谢在整个大作业完成过程中给予帮助的个人和单位。包括学术上、生活上、情感上的支持。

## 参考文献

- [1] 朱晔, 王海涛. Spring Cloud 微服务架构开发实战[M]. 电子工业出版社, 2018.
- [2] 李刚. Java 开发实战经典 (第 6 版) [M]. 清华大学出版社, 2022.
- [3] 陈文. 分布式系统原理与实践[M]. 机械工业出版社, 2021.
- [4] 杨开振. MySQL 数据库技术与应用[M]. 电子工业出版社, 2020.
- [5] 刘伟. Spring Boot 企业级开发实战[M]. 机械工业出版社, 2021.
- [6] 王振华. MyBatis 技术内幕: 深入解析 MyBatis 原理与实现[M]. 电子工业出版社, 2019.
- [7] 王磊. 微服务架构设计模式[M]. 电子工业出版社, 2020.
- [8] Pivotal Software, Inc. Spring Boot Reference Documentation [EB/OL]. <https://docs.spring.io/spring-boot/docs/current/reference/html/>, 2024.
- [9] Spring Cloud Reference Documentation [EB/OL]. <https://docs.spring.io/spring-cloud/docs/current/reference/html/>, 2024.
- [10] MyBatis-PI 官方文档[EB/OL]. <https://baomidou.com/>, 2024.
- [11] Oracle Corporation. MySQL 8.0 Reference Manual [EB/OL]. <https://dev.mysql.com/doc/refman/8.0/en/>, 2024.
- [12] Alibaba Group. Nacos Documentation [EB/OL]. <https://nacos.io/zh-cn/docs/what-is-nacos.html>, 2024.
- [13] Newman, S. Building Microservices: Designing Fine-Grained Systems. O'Reilly Media, 2015.
- [14] Richardson, C. Microservices Patterns: With examples in Java. Manning Publications, 2018.
- [15] Apifox 官方文档 [EB/OL]. <https://help.apifox.com/>, 2024.
- [16] Fowler, M. Microservices: a definition of this new architectural term [EB/OL]. <https://martinfowler.com/articles/microservices.html>, 2014.